

Mon Accord

Sunum Öncesi Anlatım Rehberi · Pre-Presentation Briefing Guide

🇹🇷 Bu rehber, web geliştirme ve yapay zeka (LLM) konusunda hiç bilgisi olmayan birinin sunum öncesinde okuyup uygulamayla ilgili gelebilecek soruların büyük çoğunluğuna cevap verebilmesi için yazıldı. Her bölüm önce Türkçe, sonra İngilizce verilmiştir.

🇬🇧 This guide was written so that someone with no background in web development or artificial intelligence (LLMs) can read it before the presentation and answer the vast majority of questions about the app. Each section is given first in Turkish, then in English.

1. Mon Accord nedir? / What is Mon Accord?

🇹🇷 Türkçe

Mon Accord, kullanıcının kendi **parfüm karışımını (formülünü)** oluşturmasına yardımcı olan bir web uygulaması mock-up'ı (taslak/prototip). Fikir şu:

- Dünyanın 6 farklı bölgesinden (İskandinavya, Doğu Asya, Güney Afrika, Akdeniz, Güney Amerika, Orta Doğu) esinlenen 12 parfüm var (her bölgenin bir **sprey** bir de **yağ** versiyonu var).
- Kullanıcı bunları "katmanlayarak" (layering) kendine özel bir koku yaratabiliyor — örneğin "2 sıkım Akdeniz spreyi + 2 damla Orta Doğu yağı".
- Uygulama, kullanıcının zevkini öğrenip ona uygun karışımları **yapay zeka ile önerir**, bu önerileri açıklar, ve kullanıcı bunları kaydedip paylaşabilir.

Ana sayfalar: **Landing** (tanıtım), **Profile Quiz** (koku kimliği anketi), **Layering Lab** (karışım atölyesi), **Vault** (kaydedilen formüller), **Shop** (mağaza mock-up'ı), **Community** (topluluk formülleri).

Önemli: Bu bir **mock-up**. Gerçek ödeme altyapısı yok; amaç konsepti ve deneyimi göstermek.

🇬🇧 English

Mon Accord is a mock-up (prototype) web application that helps users build their own **perfume blend (formula)**. The idea:

- There are 12 perfumes inspired by 6 world regions (Scandinavian, East Asia, South Africa, Mediterranean, South America, Middle East) — each region has a **spray** and an **oil** version.
- Users "layer" these to create a personal scent — e.g. "2 sprays of Mediterranean + 2 drops of Middle East oil".
- The app learns the user's taste and **uses AI to recommend** suitable blends, explains those recommendations, and lets users save and share them.

Main pages: **Landing** (intro), **Profile Quiz** (scent-identity survey), **Layering Lab**, **Vault** (saved formulas), **Shop** (store mock-up), **Community** (community formulas).

Important: This is a **mock-up**. There is no real payment system; the goal is to demonstrate the

2. "Web uygulaması" nasıl çalışır? / How does a "web app" work?

🇹🇷 Türkçe

- **Frontend (ön yüz):** Kullanıcının tarayıcıda gördüğü ve tıkladığı her şey. Mon Accord'da bu düz **JavaScript** ile yazılmış — React/Vue gibi büyük bir framework kullanılmıyor.
- **Backend (arka yüz):** Bu uygulamada klasik bir backend/veritabanı yok. Onun yerine: (1) tarayıcının kendi hafızası **localStorage** (profil, kaydedilen formüller, beğeniler burada tutulur); (2) sadece yapay zekaya güvenli istek atmak için küçük bir **Cloudflare Worker**.
- **Sayfa geçişi:** Adres çubuğundaki `#profile`, `#lab` gibi parçalar hangi sayfanın gösterileceğini belirler. Sayfa yenilenmeden içerik değişir; buna **SPA (tek sayfa uygulaması)** denir.
- **Yayın (deploy):** Frontend **GitHub Pages**'te statik site olarak; Worker ise **Cloudflare**'de barınır.

🇬🇧 English

- **Frontend:** Everything the user sees and clicks in the browser. In Mon Accord this is written in plain **JavaScript** — no big framework like React/Vue.
- **Backend:** There is no classic backend/database here. Instead: (1) the browser's own memory, **localStorage** (profile, saved formulas, likes are kept here); (2) a tiny **Cloudflare Worker** used only to send secure requests to the AI.
- **Page navigation:** The `#profile`, `#lab` parts of the URL decide which page is shown. Content changes without a full page reload — this is called an **SPA (Single Page Application)**.
- **Deployment:** The frontend is hosted on **GitHub Pages** as a static site; the Worker runs on **Cloudflare**.

3. LLM nedir? / What is an LLM?

🇹🇷 Türkçe

LLM (Büyük Dil Modeli), çok büyük miktarda metinle eğitilmiş, kendisine yazılan bir metne (**prompt**) karşılık anlamlı bir metin **üretebilen** yapay zeka modelidir. ChatGPT, Gemini, Claude bunların ürünleridir.

Siz bir talimat yazarsınız (prompt), model bunun en uygun "devamını" üretir. Doğru talimatlarla modelin **belirli bir formatta ve kişilikle** cevap vermesini sağlayabilirsiniz (buna **prompt engineering** denir).

Mon Accord, Google'ın **Gemini** modelini (hızlı/ucuz versiyon `gemini-3.1-flash-lite`) bir **API** üzerinden kullanır. API, iki yazılımın "konuşmasını" sağlayan kapıdır.

🇬🇧 English

An **LLM (Large Language Model)** is an AI model trained on huge amounts of text that can **generate** meaningful text in response to an input text (a **prompt**). ChatGPT, Gemini, and Claude are products built on this.

You write an instruction (prompt) and the model generates the most likely "continuation". With the right instructions you can make it respond in a **specific format and persona** (this is called **prompt engineering**).

Mon Accord uses Google's **Gemini** model (the fast/cheap `gemini-3.1-flash-lite`) through an **API**. An API is the doorway that lets two pieces of software "talk" to each other.

4. Uygulama LLM'e nasıl bařlanıyor? / How does the app connect to the LLM?

🇹🇷 Türkçe

Kullanıcının Tarayıcısı (Frontend) → Cloudflare Worker (aracı / proxy) API anahtarını gizler → Google Gemini API (gerçek LLM)

Neden direkt deęil? Gemini'yi kullanmak için gizli bir **API anahtarı** (bir tür şifre) gerekir. Bu anahtar tarayıcı koduna konsaydı herkes görüp çalabilirdi. Bu yüzden araya, anahtarı yalnızca kendi üzerinde (gizli) tutan küçük bir **Cloudflare Worker** koyduk.

Akış:

- Kullanıcı bir özellięi tetikler (örn. "Bana öneri ver").
- Frontend, isteęi gerçek anahtar olmadan, paylaşılan bir **Worker Token** ile Worker'a gönderir.
- Worker güvenlik kontrolleri yapar: token doğru mu, IP çok sık mı istek atıyor (**rate limit**), istek çok büyük mü, adres izinli mi (**CORS**).
- Geçerse Worker, kendi sakladığı **gizli** Gemini anahtarıyla Google'a iletir.
- Cevap aynı yoldan tarayıcıya döner.

Sonuç: **API anahtarı asla kullanıcıya görünmez**. Bu, yapay zeka entegre eden web uygulamalarının standart, güvenli yöntemidir.

🇬🇧 English

User's Browser (Frontend) → Cloudflare Worker (proxy / middleman) hides the API key → Google Gemini API (the real LLM)

Why not direct? Using Gemini requires a secret **API key** (a kind of password). If that key were placed in the browser code, anyone could see and steal it. So we put a tiny **Cloudflare Worker** in between that keeps the key only on its own (secret) side.

Flow:

- The user triggers a feature (e.g. "Give me a recommendation").
- The frontend sends the request to the Worker without the real key, using only a shared **Worker Token**.
- The Worker runs security checks: is the token valid, is the IP making too many requests (**rate limit**), is the request too large, is the origin allowed (**CORS**).
- If valid, the Worker forwards it to Google with its own **secret** Gemini key.

- The response returns to the browser the same way.

Result: **the API key is never exposed to the user.** This is the standard, secure pattern for web apps that integrate AI.

5. LLM'den ne isteniyor? / What is asked of the LLM?

🇹🇷 Türkçe

Her istekte iki şey gönderilir: (1) **Sistem talimatı** — modelin sabit "kimliği" ("Sen Mon Accord'un parfüm uzmanısın, şu 12 parfümü bilirsin, üslubun sofistike olsun"); (2) **Prompt** — kullanıcının o anki bilgileri ve istenen **çıkış formatı** (genelde belirli alanları olan bir **JSON**).

LLM'in kullanıldığı 5 ana yer:

Özellik	Sayfa	Giren	Çıkan
Koku Profili	Profile Quiz	Anket cevapları	Arketip + önerilen bölgeler + imza karışımı (JSON)
Bağlamsal Öneri	Layering Lab	Ruh hali, ortam, sezon	Formül + neden uyduğu + koku betimlemesi
Koku Simülasyonu	Layering Lab	Seçilen formülün notaları	3 aşamalı koku gelişimi (ilk/orta/dip)
Remix Önerisi	Vault/Trends	Geçmiş formüller, profil	Alışkanlığa benzer ama yeni bir formül
Formül Açıklama	Community	Topluluk formülü + profil	"Sana neden uyar/uymaz" açıklaması

JSON istemenin sebebi: Uygulama, modelin cevabını otomatik okuyup kendi tasarımıyla (kartlar, butonlar) gösterebilsin diye. Kullanıcı düz sohbet görmez; model arka planda yapılandırılmış veri üretir.

🇬🇧 English

Each request sends two things: (1) a **system instruction** — the model's fixed "identity" ("You are Mon Accord's perfume expert, you know these 12 perfumes, keep a sophisticated tone"); (2) a **prompt** — the user's current info and the requested **output format** (usually a **JSON** with specific fields).

The 5 main places the LLM is used:

Feature	Page	Input	Output
Scent Profile	Profile Quiz	Survey answers	Archetype + recommended regions + signature blend (JSON)
Contextual Rec.	Layering Lab	Mood, occasion, season	Formula + why it fits + scent description
Scent Simulation	Layering Lab	Notes of chosen formula	3-phase scent evolution (top/heart/base)
Remix Suggestion	Vault/Trends	Past formulas, profile	A formula similar to habits but with something new
Formula Explanation	Community	Community formula + profile	"Why it fits / doesn't fit you" explanation

Why JSON? So the app can automatically read the model's answer and render it with its own design (cards, buttons). The user doesn't see raw chat; the model produces structured data in the background.

6. LLM çalışmazsa? / What if the LLM fails?

🇹🇷 Türkçe

Gemini'den cevap gelmezse (internet, rate limit, anahtar sorunu) uygulama **çökmez**. Her çağrının bir **yedek planı (fallback)** vardır: bağlamsal öneri yerine önceden yazılmış hazır bir formül ("Golden Dusk" vb.), profil yerine basit kurallarla tahmin, simülasyon yerine bilinen notalardan kurulan bir cümle gösterilir. Böylece sunumda internet/API sorunu olsa bile boş ekran çıkmaz.

🇬🇧 English

If Gemini doesn't respond (internet, rate limit, key issues), the app **does not crash**. Every call has a **fallback**: instead of a contextual recommendation, a pre-written ready formula ("Golden Dusk", etc.); instead of a profile, a simple rule-based guess; instead of a simulation, a sentence built from known notes. So even if there's an internet/API problem during the presentation, there is no blank screen.

7. Sözlük / Glossary

Terim / Term	🇹🇷 Açıklama	🇬🇧 Meaning
Frontend	Tarayıcıda görünen arayüz	The interface seen in the browser
Backend	Sunucu tarafındaki iş mantığı	Server-side logic
API	İki sistemin konuştuğu arayüz	Interface where two systems talk
API key	Bir servisi kullanmak için gizli şifre	Secret password to use a service
LLM	Metin üreten büyük dil modeli	Text-generating large language model
Prompt	Modele verilen talimat metni	The instruction text given to the model
System instruction	Modelin sabit rolü/kişiliği	The model's fixed role / persona
Proxy	İsteği araya girip ileten sunucu	Server that relays the request
Rate limit	Belirli sürede istek sınırı	Request limit within a time window
CORS	İzinli adresleri sınırlayan kural	Rule limiting allowed origins
localStorage	Tarayıcının yerel veri deposu	The browser's local data store
SPA	Tek sayfa uygulaması	Single Page Application
JSON	Yapılandırılmış basit veri formatı	Simple structured data format
Deploy	Kodu canlı adrese yükleme	Publishing code to a live address

8. Muhtemel sorular / Likely questions

🇹🇷 Türkçe

- **Veritabanı var mı?** Klasik anlamda hayır; kullanıcı verisi tarayıcıda (localStorage) tutulur, sunucuda sadece küçük bir Worker var.
- **Yapay zekayı biz mi eğittik?** Hayır; Google'ın hazır Gemini modelini doğru talimatlarla (prompt engineering) kullanıyoruz. Model eğitimi değil.
- **API anahtarı çalınabilir mi?** Hayır; anahtar tarayıcıya hiç gitmez, yalnızca Cloudflare Worker'da gizli durur.
- **İnternet kesilirse?** Çalışır; her özelliğin yedek cevabı vardır.
- **Hangi model, neden?** Gemini "flash-lite"; kısa, JSON cevaplar için hızlı ve düşük maliyetli.
- **Mobilde çalışır mı?** Evet, web uygulaması olduğu için telefon tarayıcısından da açılır.
- **Hangi dille yazıldı?** JavaScript (biraz TypeScript), framework'süz; geliştirme/derleme için **Vite** kullanılıyor.
- **Nerede barınıyor?** Frontend GitHub Pages'te, Worker Cloudflare'de.

🇬🇧 English

- **Is there a database?** Not in the classic sense; user data lives in the browser (localStorage), and only a tiny Worker exists on the server side.
- **Did we train the AI?** No; we use Google's ready Gemini model with the right instructions (prompt engineering). This is not model training.
- **Can the API key be stolen?** No; the key never reaches the browser, it stays secret in the Cloudflare Worker.
- **What if the internet drops?** It works; every feature has a fallback.
- **Which model, and why?** Gemini "flash-lite"; fast and low-cost for short, JSON responses.
- **Does it work on mobile?** Yes, since it's a web app it opens in a phone browser too.
- **What language is it written in?** JavaScript (some TypeScript), no framework; **Vite** is used for development/build.
- **Where is it hosted?** Frontend on GitHub Pages, Worker on Cloudflare.

🇹🇷 Bu rehberi okuduktan sonra uygulamayı bir tarayıcıda gezip Profile Quiz → Layering Lab → Vault → Community akışını denemeniz, kavramları gözünüzde canlandırmanızı kolaylaştıracaktır.

🇬🇧 After reading this guide, walking through the app in a browser (Profile Quiz → Layering Lab → Vault → Community) will make these concepts much easier to picture.